Stacks, Queues & Monotonic Structures

Nearest smaller values, largest rectangle, sliding window minimum

Learning goals

- Know **stack** (LIFO), **queue** (FIFO), **deque** and their STL forms
- Use a stack for **matching / nesting** problems (brackets)
- **Monotonic stack**: next/previous smaller or greater element in O(n)
- Solve **largest rectangle in a histogram**
- **Monotonic deque**: sliding window min/max in O(n)
- Argue O(n) with **amortized analysis**: each element pushed/popped once

Motivation: CSES 1645 — Nearest Smaller Values

For each position, find the nearest position to its **left** with a **smaller** value (or 0 if none). $n \le 2 \cdot 10^5$.

```
i:      1  2  3  4  5  6  7  8
a:      2  5  1  4  8  3  2  5
answer: 0  1  0  3  4  3  3  7
```

Brute force scans left from every i: **$O(n^2)$** = $4 \cdot 10^{10}$. ×

Monotonic stack: **$O(n)$**. ✓

The three containers

	Insert	Remove	Peek	STL
Stack	top	top	top	`stack<T>` / `vector<T>`
Queue	back	front	front	`queue<T>`
Deque	both ends	both ends	both ends	`deque<T>`

All operations O(1).

`vector` with `push_back` / `pop_back` / `back` is a perfectly good (and slightly faster) stack — and you can iterate over it.

STL cheat sheet

```
stack<int> st;   st.push(x); st.top(); st.pop(); st.empty();
queue<int> q;    q.push(x);  q.front(); q.pop(); q.back();
deque<int> dq;   dq.push_back(x); dq.push_front(x);
                 dq.pop_back();   dq.pop_front();
                 dq.front(); dq.back(); dq[i];
```

⚠️ `pop()` returns **void** — read `top()` / `front()` first.

⚠️ `top()` / `front()` / `pop()` on an empty container = undefined behaviour.

Stack use #1: matching brackets

```
bool balanced(const string& s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') { st.push(c); continue; }
        if (st.empty()) return false;          // closer with no opener
        char o = st.top(); st.pop();
        if ((c == ')' && o != '(') || (c == ']' && o != '[') ||
            (c == '}' && o != '{')) return false;
    }
    return st.empty();                         // unclosed openers?
}
```

`{[()()]}` ✓ `([)]` × `((` ×

Stacks model nesting

Whenever the **most recent unfinished thing** must finish first:

- brackets, HTML tags, function calls (the call stack!)
- undo history
- evaluating expressions (shunting-yard)
- iterative DFS (Week 16)

Single bracket type? A counter suffices: +1 for `(`, −1 for `)`, never negative, ends at 0. Still a "stack" — just of height only.

Queues model "first come, first served"

- simulations: customers, processes, round-robin
- **BFS** (Week 16): process vertices in order of distance
- buffering a window of the last k items

```
queue<int> q;
q.push(start);
while (!q.empty()) {
    int v = q.front(); q.pop();
    // ... push neighbours
}
```

Monotonic stack: the idea

Scanning left to right, which earlier positions can ever be "the nearest smaller to the left" for some **future** i?

If j < k and a[j] ≥ a[k], then **j is useless forever**:
k is closer and at least as small.

⇒ Keep only candidates with **strictly increasing values**.
New element pops everything ≥ it, then the top is its answer.

Trace on 2 5 1 4 8 3 2 5

i	a[i]	pop	answer	stack (indices: values)
1	2	—	0	1:2
2	5	—	1	1:2 2:5
3	1	2, 1	0	3:1
4	4	—	3	3:1 4:4
5	8	—	4	3:1 4:4 5:8
6	3	5, 4	3	3:1 6:3
7	2	6	3	3:1 7:2
8	5	—	7	3:1 7:2 8:5

Code: nearest smaller to the left

```
vector<int> st;                          // indices, a[] strictly increasing
for (int i = 1; i <= n; i++) {
    while (!st.empty() && a[st.back()] >= a[i]) st.pop_back();
    ans[i] = st.empty() ? 0 : st.back();
    st.push_back(i);
}
```

Why O(n)? Each index is pushed once and popped at most once

→ ≤ 2n stack operations in total, even though one step may pop many.

That's **amortized** O(1) per element.

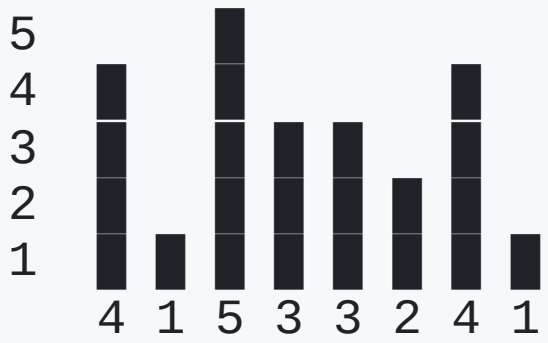
The four variants

Want	Scan	Pop while top is
previous smaller	left → right	≥ a[i]
previous greater	left → right	≤ a[i]
next smaller	right → left	≥ a[i]
next greater	right → left	≤ a[i]

- "smaller or equal" instead of "smaller": pop on `>` instead of `>=`
- Alternative for "next": scan left → right; when i pops j,
  i is j's next smaller

Largest rectangle in a histogram (CSES 1142)

Bars of heights h[i], width 1. Largest axis-aligned rectangle?




```
height 3 × width 3 = 9
height 2 × width 5 = 10  ✓ best
height 1 × width 8 = 8
```

Key fact: the optimal rectangle's height equals **some bar's height** — the lowest bar inside it. Try every bar i as "the lowest".

Histogram: extend each bar

Bar i, as the lowest bar, extends left until the first **lower** bar L[i]
and right until the first **lower** bar R[i]:

area(i) = h[i] · (R[i] − L[i] − 1)

L = previous smaller, R = next smaller → two monotonic-stack passes.

```
h:  4  1  5  3  3  2  4  1
L: -1 -1  1  1  1  1  5 -1
R:  1  8  3  5  5  7  7  8
area 4  8  5  9  9 10  4  8      max = 10
```

Histogram code

```
for (int i = 0; i < n; i++) {                    // previous smaller
    while (!st.empty() && h[st.back()] >= h[i]) st.pop_back();
    L[i] = st.empty() ? -1 : st.back();
    st.push_back(i);
}
st.clear();
for (int i = n - 1; i >= 0; i--) {               // next smaller
    while (!st.empty() && h[st.back()] >= h[i]) st.pop_back();
    R[i] = st.empty() ? n : st.back();
    st.push_back(i);
}
for (int i = 0; i < n; i++) best = max(best, h[i] * (R[i] - L[i] - 1));
```

O(n). Equal heights are fine: some bar of the group gets the full width.

Sliding window minimum

Minimum of every window of length k. n = $2 \cdot 10^5$, k arbitrary.

```
a = 1 3 -1 -3 5 3 6 7, k = 3
windows: [1 3 -1] [3 -1 -3] [-1 -3 5] [-3 5 3] [5 3 6] [3 6 7]
min:        -1       -3         -3         -3       3       3
```

- Recompute each window: O(nk)
- `multiset` of the window: O(n log k)
- **Monotonic deque: O(n)**

Monotonic deque

Deque holds indices with **increasing values**, front = window min.

For each new i:

1. Pop from the **back** while a[back] ≥ a[i]
   (they're older *and* not smaller → never the min again)
2. Push i at the back
3. Pop from the **front** if it left the window (index ≤ i − k)
4. Front is the minimum of window ending at i

Each index enters and leaves once → **O(n)** total.

Sliding window minimum code

```
deque<int> dq;                          // indices, a[] increasing
for (int i = 0; i < n; i++) {
    while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
    dq.push_back(i);
    if (dq.front() <= i - k) dq.pop_front();
    if (i >= k - 1) out(a[dq.front()]);
}
```

Maximum: flip the comparison to `<=`.

Used later for **DP optimisation** (Regionals: monotonic-queue DP).

Bonus: queue with O(1) min via two stacks

- Each stack stores pairs (value, min of stack so far)
- Push onto `in`; pop from `out`; when `out` is empty, move everything from `in` to `out`
- min = min(top of `in`.min, top of `out`.min)

Each element moves at most once → amortized O(1).
Works for any **associative** operation (gcd, max, ...), unlike the deque trick — e.g. "shortest subarray with gcd = 1" with two pointers.

Contribution technique (preview)

Sum of min over **all** subarrays, $n = 2 \cdot 10^5$:

a[i] is the minimum of subarrays that start in (L[i], i]
and end in [i, R[i]) → contributes $a[i] \cdot (i - L[i]) \cdot (R[i] - i)$.

Ties: use "< on one side, ≤ on the other"
so each subarray's minimum is counted **exactly once**.

Complexity summary

Problem	Naive	With stack/deque
bracket matching	—	O(n)
nearest smaller / greater	O(n²)	O(n) amortized
largest histogram rectangle	O(n²)	O(n)
sliding window min/max	O(nk)	O(n)
sum of subarray minimums	O(n²)	O(n)

Pitfalls

- Calling `top()` / `front()` on an **empty** container
- Storing **values** when you need **indices** (for distances, windows)
- `>=` vs `>` in the pop condition: decides strict vs non-strict answers and double counting with ties
- Forgetting to expire old indices from the deque front
- Using `int` for area / sum results — h · width can reach 10^{14}+
- Recursion-free is the point: don't rebuild the stack per query

Practice problems

- CSES 1645 — Nearest Smaller Values
- CSES 1142 — Advertisement (largest rectangle)
- CSES 3221 — Sliding Window Minimum
- Codeforces 5C — Longest Regular Bracket Sequence
- Codeforces 343B — Alternating Current
- Codeforces 547B — Mike and Feet
- Codeforces 1407D — Discrete Centrifugal Jumps
- Kattis evenup — Even Up Solitaire
- LeetCode 739 / 84 / 239 — Daily Temperatures, Largest Rectangle, Sliding Window Max

Summary

- Stack = nesting, queue = order of arrival, deque = both
- **Monotonic stack**: discard dominated candidates → nearest smaller/greater in O(n)
- Histogram: each bar extends to its nearest lower neighbours
- **Monotonic deque**: sliding window min/max in O(n)
- Amortized analysis: count pushes and pops per element, not per step
- Next week: **priority queues, ordered sets/maps, policy-based DS**